MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 11

REPORT ON: TASK REPORT

TASK NO: 01-03

TASK 1 – DATA SOURCE DESIGN & LIST STRUCTURE PLANNING

Introduction

This task focuses on designing the data model and layout structure for a scrollable list UI in Jetpack Compose. Planning the data structure and UI layout helps developers organize information before implementing the feature in code.

Data Model

Theme: **Daily Motivational Quotes**

```
data class Quote(
    val title: String,
    val author: String,
    val imageResId: Int,
    val description: String,
    val isFavourite: Boolean
)
```

Sample Dataset

Title	Author	Image	Description	Favourite
Believe in Yourself	Unknown	R.drawable.quote1	Motivation for success	false
Never Give Up	Winston Churchill	R.drawable.quote2	Encourages persistence	true
Dream Big	Walt Disney	R.drawable.quote3	Inspiration for ambition	false
Stay Positive	Dalai Lama	R.drawable.quote4	Encourages positive thinking	false
Work Hard	Elon Musk	R.drawable.quote5	Hard work leads to success	true
Keep Learning	Albert Einstein	R.drawable.quote6	Learning never stops	false
Success Mindset	Tony Robbins	R.drawable.quote7	Focus on growth	false
Be Creative	Steve Jobs	R.drawable.quote8	Creativity drives innovation	false
Courage Matters	Nelson Mandela	R.drawable.quote9	Leadership inspiration	false
Think Different	Apple	R.drawable.quote10	Innovation mindset	true

UI Layout Plan

Wireframe

```
-----------------------------------
| Image | Quote Title              |
|       | Author Name              |
|       | Short Description        |
|       | ♥ Favourite Icon         |
-----------------------------------
```

Compose Components Used

- LazyColumn
- Card
- Row
- Column
- Image
- Text
- Icon

Padding and spacing will be added using **Modifier.padding()**.

Interaction Plan

When the user taps a quote item, the application displays detailed information about the quote. When the favourite icon is tapped, the **isFavourite state toggles**, updating the UI immediately.

Reflection

Designing the data model and UI layout before coding helps developers clearly understand the structure of their application. This planning reduces errors during implementation and ensures the UI aligns properly with the available data.

TASK 2 – RESPONSIVE LIST & IMAGE LOADING STRATEGY

Responsive Layout Plan

Phone Layout

- Single column layout using **LazyColumn**

Tablet Layout

- Two-column grid using **LazyVerticalGrid**

This allows the application to adapt to different screen sizes and orientations.

Image Loading Strategy

Images will be loaded using **local drawable resources**.

Example:

```
Image(
    painter = painterResource(id = quote.imageResId),
    contentDescription = quote.title
)
```

For internet images, the **Coil library** can be used.

Example:

```
AsyncImage(
    model = imageUrl,
    contentDescription = "Quote Image"
)
```

Placeholder Strategy

- Display placeholder image while loading
- Show fallback image if loading fails

Performance Considerations

1. Use **LazyColumn** so only visible items render.
2. Optimize image sizes.
3. Avoid loading large drawables.

Resource Management

Target image size: **200 × 200 dp**

Large datasets will use lazy loading so only visible items are loaded.

Reflection

Image-heavy lists can impact performance if not optimized. Responsive layouts increase complexity because the UI must adjust across devices. Efficient image loading and lazy rendering ensure smooth performance.

TASK 3 – UI ITEM INTERACTION DESIGN & STATEFUL FAVOURITE FEATURE

Favourite Feature

Each list item contains a **favourite icon** that toggles the favourite state.

Example UI

Quote Title ♥
Author Name
Short Description

State Management Plan

Example state variable

var isFavourite by remember { mutableStateOf(false) }

State can also be managed through a **ViewModel** to maintain state during scrolling.

Item Expansion Plan

When a user taps an item, additional content expands.

Example:

var expanded by remember { mutableStateOf(false) }

```
if (expanded) {
    Text(quote.description)
}
```

Animation example:

animateContentSize()

Edge Cases

Scenario	Solution
Rapid favourite toggling	Prevent multiple quick updates
LazyColumn reusing items	Store state in ViewModel
Screen rotation	Use rememberSaveable

Reflection

Interactive list items increase application complexity because state must be managed properly during recomposition and scrolling. Proper state handling ensures consistent user experience.

TASK 4 – CAPSTONE: FULL SCROLLABLE LIST FEATURE

Feature Specification

Feature: **Motivational Quotes List**

Users can browse a list of motivational quotes with images and descriptions. They can mark quotes as favourites and expand items to view more details.

Data & UI Documentation

Includes:

- Quote data model
- Dataset of 10 items
- Scrollable LazyColumn UI
- Favourite toggle functionality

- Expandable list items

Flow Diagram

Quote Data Source

|

v

Scrollable LazyColumn UI

|

User Interaction

|

Toggle Favourite / Expand Item

|

State Update

|

UI Recomposition

Implementation Timeline

Step	1
Create data class and dataset	
Step	2
Build LazyColumn UI	
Step	3
Add images and responsive layout	
Step	4
Add favourite and expansion features	
Step	5
Test performance and edge cases	

Dependencies

- Jetpack Compose
- Coil (Image Loading)
- Compose Animation APIs

Reflection

Building a scrollable list with interactive features requires careful planning of data structures, UI layout, and state management. Combining responsive layouts, image loading strategies, and interactive components makes the feature more complex but improves user experience. Future improvements may include search, filtering, and offline caching.